

REPUBLIQUE DU SENEGAL



Un peuple-un but-une foi

Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation

Direction de l'Enseignement Supérieur Privé

Institut Supérieur d'Informatique

ISI

**SUPERVISION RÉSEAU AVEC NETFLOW/SFLOW COUPLÉE À ELK
ET NTOPNG POUR L'ANALYSE ET LA SÉCURISATION DU TRAFIC**

Présenté par :

M. Ayoub AWWALE BILE

Sous la direction de

M. Ahmed Ould KHALIFA



Année Académique : 2025 -2026

Sommaire

Introduction	1
Problématique.....	1
Objectifs.....	1
Cas d'usage métier couverts.....	2
I. Présentation de la solution	2
Contexte et objectifs	2
Présentation des composants	3
II. Cadre théorique.....	4
Le protocole NetFlow	4
Le protocole sFlow	4
La suite Elastic (ELK)	5
NtopNG et l'écosystème ntop	6
Positionnement face aux autres outils de supervision	6
III. Architecture de la solution.....	7
Prérequis	7
Topologie du laboratoire	8
Sécurité de l'architecture.....	8
IV. Mise en œuvre pratique du laboratoire	9
Installation de NtopNG.....	9
Choix de la chaîne de collecte NetFlow	9
Installation et configuration de netflow2ng.....	10
Génération et export des flux depuis client-test	14
Installation et configuration de la pile ELK	18
Export via GNS3 (routeur et switch réels)	26
VI. Résultats et tests	28
Indicateurs de performance observés	28
VII. Sécurité et détection d'anomalies.....	29
Test réalisé en laboratoire.....	29
Apports et limites observés.....	29
Comparaison avec les solutions commerciales	30
Conformité et aspects réglementaires	30
VIII. Dimensionnement et bonnes pratiques	30
Bonnes pratiques de déploiement	31
Conclusion.....	32
Recommandations pour un déploiement en production	32

Mot de fin	32
Annexe — Glossaire et références	33
Glossaire des termes techniques.....	33
Tableau récapitulatif des ports utilisés.....	33
Références	34

Introduction

Ce rapport présente la mise en place d'une solution de **supervision réseau** basée sur l'analyse des flux exportés par les équipements actifs, au moyen des protocoles **NetFlow** et **sFlow**, couplée à deux chaînes de traitement complémentaires : la suite **Elastic (ELK)** et la plateforme open source **NtopNG**.

Dans un contexte où les infrastructures réseau des entreprises supportent un nombre croissant d'applications critiques, la capacité à observer, comprendre et anticiper le comportement du trafic réseau est devenue un enjeu opérationnel majeur. J'ai réalisé un projet complet de supervision réseau, depuis la compréhension des protocoles d'export de flux jusqu'à la mise en œuvre pratique d'un laboratoire fonctionnel.

Ce rapport détaille l'ensemble des étapes réalisées : présentation théorique des protocoles et outils, conception de l'architecture, mise en œuvre du laboratoire de test (machines virtuelles, **GNS3**), aspects de sécurité, et résultats obtenus.

Problématique

La question centrale à laquelle ce travail tente de répondre peut se formuler ainsi : comment concevoir et déployer une architecture de supervision réseau capable de collecter, centraliser, analyser et visualiser en quasi temps réel les flux **NetFlow/sFlow** d'une infrastructure, tout en combinant les forces complémentaires d'une solution généraliste (**ELK**) et d'une solution spécialisée dans l'analyse de trafic (**NtopNG**) ?

Cette problématique se décline en plusieurs questions opérationnelles :

- Quels sont les mécanismes techniques sous-jacents aux protocoles NetFlow, sFlow et IPFIX, et quelles sont leurs différences fondamentales ?
- Comment architecturer une chaîne de collecte, de stockage et de visualisation robuste et évolutive ?
- Quels sont les rôles respectifs de la pile ELK et de NtopNG, et en quoi sont-ils complémentaires plutôt que redondants ?
- Quelles difficultés pratiques se posent lors du déploiement en environnement réel ?

Objectifs

Ce travail poursuit les objectifs suivants :

- Présenter le cadre théorique des protocoles de supervision par flux et de leurs cas d'usage.
- Décrire l'architecture fonctionnelle et technique d'une solution combinant ELK et NtopNG.
- Détailler la mise en œuvre pratique d'un laboratoire de test, depuis l'installation jusqu'à la configuration.
- Analyser les résultats obtenus et formuler des recommandations pour un déploiement en production.

Cas d'usage métier couverts

Une solution de supervision réseau par analyse de flux répond à plusieurs besoins concrets rencontrés en entreprise :

- **Planification de capacité** : anticiper la saturation d'un lien réseau avant qu'elle n'affecte les utilisateurs.
- **Détection de sécurité de premier niveau** : repérer des scans, des tentatives d'exfiltration ou des communications suspectes.
- **Contrôle d'usage et facturation interne** : refacturer la bande passante entre directions ou filiales d'une même organisation.
- **Diagnostic d'incident** : identifier rapidement l'origine d'une saturation ou d'une latence applicative signalée par les utilisateurs.
- **Conformité et audit** : assurer la traçabilité des flux pour répondre à des exigences d'audit interne ou réglementaire.

I. Présentation de la solution

Contexte et objectifs

La supervision réseau traditionnelle repose sur deux familles d'outils : la supervision par interrogation (**SNMP**), qui fournit des compteurs agrégés sans granularité applicative, et la capture intégrale de paquets, qui offre une visibilité totale mais à un coût de traitement très élevé. Entre ces deux approches se positionne la **supervision par flux**, qui consiste à observer des agrégats de paquets partageant les mêmes caractéristiques plutôt que chaque paquet individuellement.

L'objectif principal de ce projet est de déployer une solution de supervision **complète et opérationnelle**, capable de :

- Collecter et centraliser les flux NetFlow/sFlow provenant de différents équipements du réseau
- Analyser et visualiser le trafic en temps réel
- Détecter rapidement les incidents et anomalies de sécurité
- Présenter les données sous forme de tableaux de bord clairs et exploitables

Présentation des composants

La solution déployée repose sur plusieurs composants qui fonctionnent ensemble pour assurer une supervision complète et efficace du trafic réseau.

Composant	Rôle
NetFlow / sFlow	Protocoles d'export de flux exécutés par les équipements réseau (routeurs, switches)
softflowd / netflow2ng	Génération de flux NetFlow (softflowd) et collecte/republication vers ntopng via ZMQ (netflow2ng), alternative gratuite à nProbe
NtopNG	Plateforme de visualisation temps réel et d'identification applicative (nDPI)
Logstash	Pipeline d'ingestion qui décode et enrichit les flux avant indexation
Elasticsearch	Indexation et stockage des flux sous forme de documents interrogeables
Kibana	Tableaux de bord interactifs, recherche libre et alerting avancé

Tableau 1 — Composants principaux de la solution déployée

NetFlow, développé par Cisco, agrège les paquets en flux définis par un septuplet de clés (IP source/destination, ports, protocole, ToS, interface) et exporte un résumé de chaque flux vers un collecteur, généralement en UDP sur le port **2055**. C'est un protocole « stateful » qui maintient un état complet pour chaque flux observé.

sFlow, à l'inverse, ne maintient aucun état (« stateless ») : il combine un échantillonnage statistique des paquets avec un relevé périodique des compteurs d'interface, ce qui réduit fortement la charge sur l'équipement. Il est exporté en UDP sur le port **6343**.

La suite **ELK (Elasticsearch, Logstash, Kibana)** constitue une plateforme généraliste de gestion de logs et de métriques, tandis que **NtopNG** est une solution spécialisée dans l'analyse de trafic réseau avec une interface web prête à l'emploi, capable de recevoir les flux via une sonde dédiée comme **nProbe** (commerciale) en environnement de production, ou via une alternative ouverte comme **netflow2ng**, retenue pour ce laboratoire.

Pourquoi combiner ELK et NtopNG ?

NtopNG offre une mise en œuvre rapide et une identification applicative très forte grâce à son moteur **nDPI**, ce qui en fait un outil de premier niveau pour le diagnostic immédiat. ELK, de son côté, permet une corrélation multi-sources et une personnalisation poussée des tableaux de bord, ce qui en fait une plateforme de corrélation et de reporting à plus long terme. Les deux chaînes sont alimentées par les mêmes flux, dupliqués à la source, et fonctionnent de manière **totalemtent indépendante** : la panne de l'une n'affecte pas l'autre.

II. Cadre théorique

Le protocole NetFlow

NetFlow a été développé par Cisco Systems au milieu des années 1990. Un équipement compatible observe le trafic transitant par ses interfaces et regroupe les paquets en flux unidirectionnels définis par un **septuplet de clés** : adresse IP source, adresse IP destination, port source, port destination, protocole, type de service (ToS) et interface d'entrée.

Pour chaque flux identifié, l'équipement maintient une entrée dans une table de flux (cache NetFlow) qui comptabilise le nombre de paquets, le volume d'octets, ainsi que les horodatages de début et de fin. Lorsqu'un flux est considéré comme terminé, l'équipement exporte l'enregistrement vers un collecteur, généralement via **UDP sur le port 2055**.

Version	Caractéristiques principales
v5	Format fixe et figé, IPv4 uniquement. Version historiquement la plus répandue, simple mais sans extensibilité.
v9	Introduction de modèles (templates) permettant de définir dynamiquement les champs exportés. Support IPv6, VLAN, MPLS.
IPFIX	Standard IETF (RFC 7011), dérivé de la v9 mais indépendant de Cisco, avec des champs extensibles enregistrés auprès de l'IANA.

Tableau 1 — Évolution des versions du protocole NetFlow

Le protocole sFlow

sFlow (« sampled Flow ») est un protocole concurrent de NetFlow, développé par InMon Corporation. Contrairement à NetFlow, qui maintient un état complet pour chaque flux, sFlow repose sur un principe d'**échantillonnage statistique**, combinant l'échantillonnage de paquets (1 paquet sur N) et le relevé périodique des compteurs d'interface.

Cette approche a une conséquence directe : sFlow est **stateless** et n'a besoin d'aucune mémoire dédiée par flux, ce qui réduit fortement la consommation de ressources sur l'équipement — un avantage décisif sur des liens à très haut débit où NetFlow pourrait saturer le plan de contrôle. Les datagrammes sont envoyés en **UDP sur le port 6343**.

Critère	NetFlow / IPFIX	sFlow
Méthode	Exhaustive (stateful)	Échantillonnage (stateless)
Précision	Exacte	Statistique
Charge équipement	Modérée à élevée	Très faible
Port UDP usuel	2055	6343
Cas d'usage	Réseaux d'entreprise, facturation	Cœurs réseau très haut débit

Tableau 2 — Comparaison synthétique NetFlow/IPFIX vs sFlow

Astuce : NetFlow et sFlow ne sont pas mutuellement exclusifs : une architecture de collecte bien conçue peut ingérer les deux protocoles simultanément, en s'adaptant à l'hétérogénéité des équipements présents dans le parc réseau.

La suite Elastic (ELK)

La pile **ELK** désigne l'association de trois logiciels open source édités par Elastic NV : **Elasticsearch** (moteur de recherche et d'indexation), **Logstash** (pipeline de collecte et de transformation), et **Kibana** (interface de visualisation).

Elasticsearch

Elasticsearch est un moteur de recherche distribué construit sur Apache Lucene. Il stocke les données sous forme de documents JSON organisés en index, et permet des recherches complexes en quelques millisecondes même sur des volumes de plusieurs millions de documents. Chaque enregistrement de flux NetFlow/sFlow devient un document indexé.

Logstash

Logstash est un pipeline structuré en trois étapes : entrée, filtrage/transformation, et sortie. Pour l'ingestion de flux, Logstash dispose d'un plugin d'entrée **netflow** capable de décoder NetFlow v5, v9, IPFIX et sFlow v5. Une alternative répandue est **ElastiFlow**, qui fournit des pipelines et tableaux de bord préconstruits, simplifiant considérablement la mise en œuvre.

Kibana

Kibana permet de construire des tableaux de bord interactifs (graphiques temporels, cartes de chaleur, diagrammes de Sankey) à partir des données indexées dans Elasticsearch, ainsi que de définir des alertes basées sur des seuils ou des anomalies statistiques.

NtopNG et l'écosystème ntop

NtopNG est une plateforme open source d'analyse de trafic réseau, héritière de l'outil historique « ntop ». Elle se distingue de la pile ELK par sa spécialisation : conçue spécifiquement pour la supervision réseau, avec une interface web prête à l'emploi et un moteur d'alerte intégré.

- **nProbe** : sonde NetFlow/IPFIX/sFlow qui collecte les flux et les retransmet normalisés vers ntopng via un bus ZMQ. Cette sonde nécessite une licence ntop, même pour un usage basique ; le laboratoire lui substitue **netflow2ng**, un collecteur ouvert reproduisant le même rôle de proxy ZMQ (cf. § Mise en œuvre).
- **nDPI** : moteur d'inspection profonde (Deep Packet Inspection) utilisé pour l'identification automatique des applications.
- **ClickHouse** : base de données orientée colonnes utilisée pour historiser les flux sur la durée tout en conservant des temps de requête rapides.

Positionnement face aux autres outils de supervision

Il est utile de situer NetFlow/sFlow, ELK et NtopNG par rapport au paysage plus large des outils de supervision réseau. Des solutions comme **Zabbix**, **Centreon** ou **LibreNMS** restent très répandues pour la supervision de disponibilité (SNMP), mais ne fournissent aucune granularité sur la nature du trafic.

L'écosystème **Grafana/Prometheus**, très populaire dans les environnements DevOps, excelle dans la visualisation de métriques mais ne dispose pas nativement d'un moteur de décodage NetFlow/sFlow comparable à Logstash ou nProbe. Les outils de capture comme **Wireshark** ou **Zeek** offrent une richesse d'analyse inégalée, mais à un coût de ressources incompatible avec une supervision permanente à l'échelle de l'entreprise.

Le choix de NetFlow/sFlow couplé à ELK et NtopNG constitue ainsi un compromis délibéré, situé entre la supervision SNMP générique et la capture de paquets complète — répondant aux besoins de visibilité applicative suffisante et de coût de ressources maîtrisé.

III. Architecture de la solution

L'architecture proposée s'organise en quatre couches : la couche de génération/export des flux (les équipements réseau), la couche de collecte (**ntopng**), la couche de stockage et d'indexation (**ClickHouse** pour ntopng, **Elasticsearch** pour la pile ELK), et la couche de visualisation (interface web ntopng et **Kibana**).

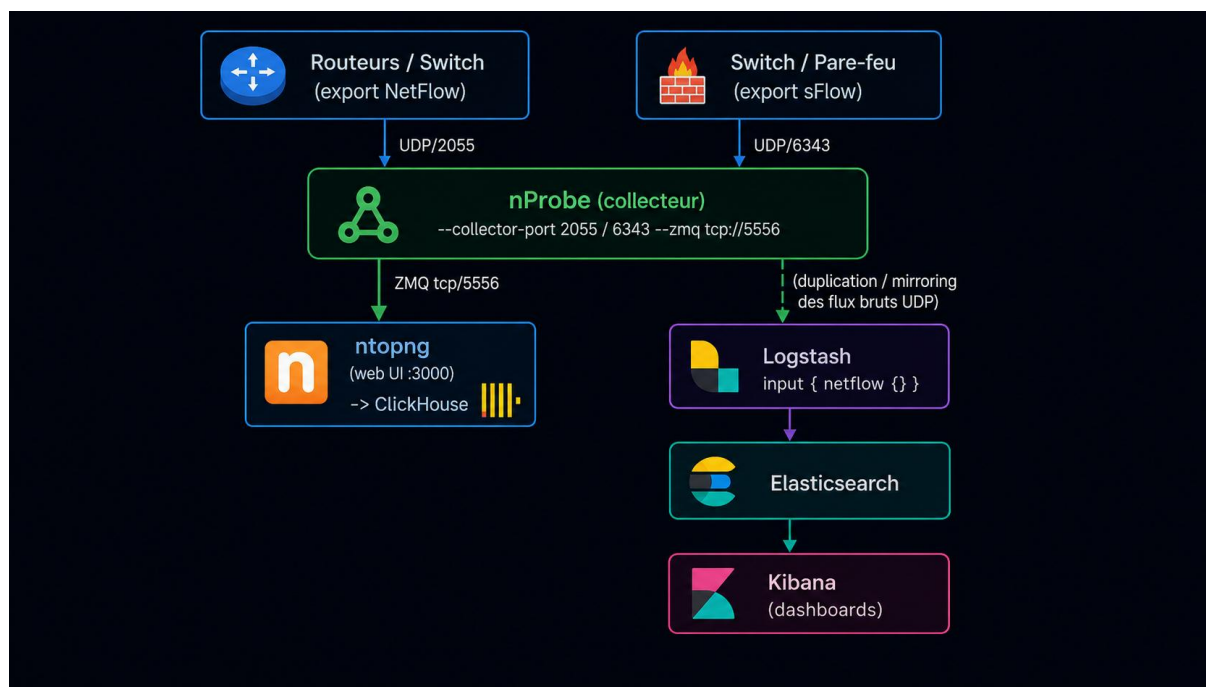


Figure 1 — Architecture de bout en bout : flux NetFlow/sFlow vers ntopng et vers la chaîne ELK

Astuce : ce schéma représente l'architecture de référence telle que recommandée par ntop pour un déploiement de production, avec nProbe comme sonde de collecte dédiée. Pour le laboratoire, cette sonde a été remplacée par netflow2ng, un collecteur open source assurant le même rôle de proxy ZMQ vers ntopng, nProbe nécessitant une licence même pour un usage basique (cf. § Mise en œuvre).

Prérequis

Avant de commencer le déploiement, les conditions préalables suivantes doivent être remplies :

- Vérifier que les ports requis sont ouverts : **UDP 2055** (NetFlow), **UDP 6343** (sFlow), **TCP 5556** (ZMQ netflow2ng → ntopng), **TCP 3000** (ntopng), **TCP 9200** (Elasticsearch), **TCP 5601** (Kibana).
- Disposer de trois machines virtuelles sous Ubuntu Server 22.04 LTS, sans interface graphique.
- Prévoir un hôte disposant d'au moins 16 Go de RAM pour exécuter confortablement les trois VM.
- Vérifier la connectivité réseau entre les VM avant de commencer toute installation.

Topologie du laboratoire

Le laboratoire repose sur **trois machines virtuelles** sous VMware Workstation Pro, complétées par **GNS3** pour virtualiser un routeur Cisco et un switch, reliés au réseau des VM via un adaptateur **VMnet Host-only (VMnet2)**.

VM	Rôle	Adresse IP	Ressources
VM1 — ntop-srv	netflow2ng + ntopng + ClickHouse	192.168.10.10	2 vCPU / 3 Go RAM
VM2 — elk-srv	Elasticsearch + Logstash + Kibana	192.168.10.20	2-3 vCPU / 6-7 Go RAM
VM3 — client-test	Génération de trafic et export simulé	192.168.10.50	1 vCPU / 1-2 Go RAM

Tableau 2 — Topologie des machines virtuelles du laboratoire

Astuce : les trois VM sont installées sous Ubuntu Server (sans environnement de bureau) plutôt que sous Windows : les outils de collecte s'installent nativement sous Linux, et une installation sans interface graphique consomme nettement moins de RAM, ce qui est précieux sur un hôte à 16 Go partagés entre trois VM.

Sécurité de l'architecture

Plusieurs mesures de sécurisation sont intégrées dès la conception :

- **Isolation réseau :** les flux NetFlow/sFlow circulent sur un réseau dédié, non routable depuis l'extérieur.
- **Filtrage des collecteurs :** restriction des adresses IP autorisées à envoyer des flux UDP vers ntopng, via les règles de pare-feu de la VM ntop-srv.
- **Accès web restreint :** interfaces ntopng et Kibana exposées via un reverse proxy avec authentification.
- **Gestion des comptes :** changement systématique des comptes par défaut (admin/admin) dès la première connexion.

IV. Mise en œuvre pratique du laboratoire

Cette partie détaille la mise en œuvre concrète du laboratoire, depuis l'installation de **ntopng** jusqu'à la configuration complète de la chaîne **ELK** et la mise en place d'une collecte NetFlow réelle et fonctionnelle.

Installation de NtopNG

- Étape 1 — Installation des paquets ntop

```
curl -s https://packages.ntop.org/apt-stable/22.04/all/apt-ntop-stable.deb \
-o /tmp/apt-ntop-stable.deb
```

```
root@ntopser:~# curl -s https://packages.ntop.org/apt-stable/22.04/all/apt-ntop-stable.deb -o /tmp/apt-ntop-stable.deb
```

```
sudo apt install -y /tmp/apt-ntop-stable.deb
```

```
root@ntopser:~# apt install -y /tmp/apt-ntop-stable.deb
```

```
sudo apt update
```

```
root@ntopser:~# apt update
```

```
sudo apt install -y ntopng
```

```
root@ntopser:~# apt install -y ntopng
```

Choix de la chaîne de collecte NetFlow

La sonde **nProbe**, recommandée par ntop pour les déploiements de production (cf. § Architecture), nécessite une licence commerciale même pour un usage basique en collecteur — une tentative de lancement direct se solde par l'erreur « **Invalid or missing license** ». Une première alternative testée a consisté à faire écouter ntopng directement sur les ports NetFlow et sFlow via la syntaxe **-i=netflow://2055**, mais cette fonctionnalité s'est révélée non opérationnelle sur la version de ntopng utilisée (6.6), l'interface échouant systématiquement à l'ouverture avec l'erreur « **Unable to open interface netflow://2055** » dans les journaux systemd.

La solution finalement retenue repose sur **netflow2ng**, un collecteur NetFlow/IPFIX open source qui reproduit le rôle de proxy de nProbe (réception des flux UDP puis republication via un bus **ZMQ** vers ntopng), sans nécessiter de licence. Cette chaîne complète a été validée de bout en bout sur le laboratoire.

Chaîne de collecte retenue : **softflowd** (sur client-test, génère et exporte les flux NetFlow v9) → **netflow2ng** (sur ntop-srv, reçoit les flux UDP/2055 et les republie en ZMQ) → **ntopng** (se connecte au flux ZMQ pour la visualisation).

Installation et configuration de netflow2ng

netflow2ng est écrit en Go et doit être compilé depuis les sources. Cette compilation nécessite une version récente du compilateur Go (la version fournie par les dépôts Ubuntu 22.04 est trop ancienne pour le format du fichier go.mod du projet).

➤ Étape 1 — Installation d'une version récente de Go

```
sudo apt install -y golang-go git
```

```
root@ntopser:~# apt install -y golang-go git
```

```
cd /tmp
```

```
root@ntopser:~# cd /tmp
```

```
wget https://go.dev/dl/go1.23.4.linux-amd64.tar.gz
```

```
root@ntopser:/tmp# wget https://go.dev/dl/go1.23.4.linux-amd64.tar.gz
--2026-06-26 11:06:51-- https://go.dev/dl/go1.23.4.linux-amd64.tar.gz
Résolution de go.dev (go.dev)... 216.239.36.21, 216.239.34.21, 216.239.38.21, ...
Connexion à go.dev (go.dev)[216.239.36.21]:443... connecté.
requête HTTP transmise, en attente de la réponse... 302 Found
Emplacement : https://dl.google.com/go/go1.23.4.linux-amd64.tar.gz [suivant]
--2026-06-26 11:06:51-- https://dl.google.com/go/go1.23.4.linux-amd64.tar.gz
Résolution de dl.google.com (dl.google.com)... 142.250.129.136, 142.250.129.190, 142.250.129.91, ...
Connexion à dl.google.com (dl.google.com)[142.250.129.136]:443... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 73645095 (70M) [application/x-gzip]
Enregistre : 'go1.23.4.linux-amd64.tar.gz'

go1.23.4.linux-amd64.tar.gz 100%[=====] 70,23M 2,23MB/s ds 27s
2026-06-26 11:07:19 (2,58 MB/s) - 'go1.23.4.linux-amd64.tar.gz' enregistré [73645095/73645095]
```

```
sudo tar -C /usr/local -xzf go1.23.4.linux-amd64.tar.gz
```

```
echo 'export PATH=$PATH:/usr/local/go/bin' | sudo tee -a /etc/profile.d/golang.sh
```

```
source /etc/profile.d/golang.sh
```

```
go version
```

```
root@ntopser:/tmp# tar -C /usr/local -xzf go1.23.4.linux-amd64.tar.gz
root@ntopser:/tmp# echo 'export PATH=$PATH:/usr/local/go/bin' | sudo tee -a /etc/profile.d/golang.sh
export PATH=$PATH:/usr/local/go/bin
root@ntopser:/tmp# source /etc/profile.d/golang.sh
root@ntopser:/tmp# go version
go version go1.23.4 linux/amd64
root@ntopser:/tmp#
```

➤ Étape 2 — Installation des dépendances de compilation

La compilation requiert également le compilateur **Protocol Buffers** (protoc), son plugin Go, et la bibliothèque système **libzmq** (utilisée pour la communication ZMQ avec ntopng) :

```
sudo apt install -y protobuf-compiler libzmq3-dev pkg-config
```

```
root@ntopser:/tmp/netflow2ng# apt install -y libzmq3-dev pkg-config
```

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
```

```
echo 'export PATH=$PATH:$(go env GOPATH)/bin' | sudo tee -a /etc/profile.d/golang.sh
```

```
source /etc/profile.d/golang.sh
```

```
root@ntopser:/tmp# go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
# google.golang.org/protobuf/reflect/protoreflect
/root/go/pkg/mod/google.golang.org/protobuf@v1.36.11/reflect/protoreflect/value_unsafe.go:65:59: undefined: unsafe.SliceData
/root/go/pkg/mod/google.golang.org/protobuf@v1.36.11/reflect/protoreflect/value_unsafe.go:68:58: undefined: unsafe.SliceData
/root/go/pkg/mod/google.golang.org/protobuf@v1.36.11/reflect/protoreflect/value_unsafe.go:76:16: undefined: unsafe.SliceData
note: module requires Go 1.23
```

```
root@ntopser:/tmp# echo 'export PATH=$PATH:$(go env GOPATH)/bin' | sudo tee -a /etc/profile.d/golang.sh
export PATH=$PATH:$(go env GOPATH)/bin
root@ntopser:/tmp#
```

```
root@ntopser:/tmp# source /etc/profile.d/golang.sh
root@ntopser:/tmp#
```

➤ Étape 3 — Compilation et installation de netflow2ng

```
cd /tmp
```

```
root@ntopser:~# cd /tmp
```

```
git clone https://github.com/synfinatic/netflow2ng.git
```

```
root@ntopser:/tmp# git clone https://github.com/synfinatic/netflow2ng.git
Clonage dans 'netflow2ng'...
remote: Enumerating objects: 974, done.
remote: Counting objects: 100% (324/324), done.
remote: Compressing objects: 100% (134/134), done.
remote: Total 974 (delta 249), reused 212 (delta 190), pack-reused 650 (from 2)
Réception d'objets: 100% (974/974), 376.60 Kio | 976.00 Kio/s, fait.
Résolution des deltas: 100% (505/505), fait.
```

```
cd netflow2ng
```

```
root@ntopser:/tmp# cd /tmp/netflow2ng
```

```
make
```

```
root@ntopser:/tmp/netflow2ng# make
mkdir -p dist/
go mod download
protoc --proto_path=/root/go/pkg/mod/github.com/netsampler/goflow2/v2@v2.2.6 \
  --proto_path=./proto --go_out=./proto \
  --go_opt=paths=source_relative \
  extended_flow.proto
Modifying import path in extended_flow.pb.go to add 'v2' to the go package path.
sed -i 's|pb "github.com/netsampler/goflow2/pb"|pb "github.com/netsampler/goflow2/v2/pb"|g' proto/extended_flow.pb.go
go build -ldflags='-X "main.Version=0.2.2" -X "main.Delta="' -X "main.Buildinfos=2026-06-28T08:56:59+0000" -X "main.Tag=v0.2.2-35-g8266e56" -X "main.CommitID=8266e56c939371b756f9567113f64e5778c72162"' -o dist/netflow2ng-0.2.2 ./cmd/...
```

```
sudo cp dist/netflow2ng-* /usr/local/bin/netflow2ng
```

```
sudo chmod +x /usr/local/bin/netflow2ng
```

```
root@ntopser:/tmp/netflow2ng# cp dist/netflow2ng-0.2.2 /usr/local/bin/netflow2ng
root@ntopser:/tmp/netflow2ng# chmod +x /usr/local/bin/netflow2ng
root@ntopser:/tmp/netflow2ng# netflow2ng --help
Usage: netflow2ng [flags]

NetFlow v9/IPFIX Proxy for ntopng

Flags:
  -h, --help                Show context-sensitive help.
  -a, --listen="0.0.0.0:2055" NetFlow/IPFIX listen address:port
  --reuse                    Enable SO_REUSEPORT for NetFlow/IPFIX listen port
  -m, --metrics="0.0.0.0:8080" Metrics listen address
  -z, --listen-zmq="tcp://*:5556" proto://IP:Port to listen on for ZMQ connections
  --topic="flow"             ZMQ Topic
  --source-id=0              NetFlow SourceId (0-255)
  -f, --format="tlv"          Output format [tlv|json|jcompress|proto] for ZMQ.
  -w, --workers=2            Number of NetFlow workers
  -l, --log-level="info"      Log level [error|warn|info|debug|trace]
  --log-format="default"      Log format [default|json]
  -v, --version              Print version and copyright info
```

Astuce : le binaire compilé accepte l'option `--help` pour lister ses paramètres : `--listen` (port d'écoute NetFlow/IPFIX, par défaut 0.0.0.0:2055), `--listen-zmq` (point de publication ZMQ vers ntopng, par défaut tcp://*:5556), et `--format` (tlv par défaut, requis par les versions récentes de ntopng en édition Community).

➤ Étape 4 — Service systemd pour netflow2ng

Afin que le collecteur survive aux redémarrages, il est encapsulé dans un service systemd :

```
# /etc/systemd/system/netflow2ng.service
```

```
[Unit]
```

```
Description=netflow2ng - NetFlow v9/IPFIX proxy for ntopng
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
ExecStart=/usr/local/bin/netflow2ng --listen 0.0.0.0:2055 --listen-zmq tcp://*:5556
```

```
Restart=on-failure
```

```
RestartSec=5
```

```
[Install]
```

```
WantedBy=multi-user.target
```



```
root@ntopser: /tmp/netflow2ng
root@ntopser: /tmp/netflow2ng# nano /etc/systemd/system/netflow2ng.service
```

```
root@ntopser: /tmp/netflow2ng
GNU nano 6.2 /etc/systemd/system/netflow2ng.service *
[Unit]
Description=netflow2ng - NetFlow v9/IPFIX proxy for ntopng
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/netflow2ng --listen 0.0.0.0:2055 --listen-zmq tcp://*:5556
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

sudo systemctl daemon-reload

sudo systemctl enable netflow2ng

sudo systemctl start netflow2ng

sudo systemctl status netflow2ng

```
root@ntopser: /tmp/netflow2ng# systemctl daemon-reload
root@ntopser: /tmp/netflow2ng# systemctl enable netflow2ng
Created symlink /etc/systemd/system/multi-user.target.wants/netflow2ng.service → /etc/systemd/system/netflow2ng.service.
root@ntopser: /tmp/netflow2ng# systemctl start netflow2ng
root@ntopser: /tmp/netflow2ng# systemctl status netflow2ng
● netflow2ng.service - netflow2ng - NetFlow v9/IPFIX proxy for ntopng
   Loaded: loaded (/etc/systemd/system/netflow2ng.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-06-26 12:06:26 GMT; 30s ago
     Main PID: 65402 (netflow2ng)
        Tasks: 9 (limit: 4540)
      Memory: 3.2M
         CPU: 21ms
    CGroup: /system.slice/netflow2ng.service
            └─65402 /usr/local/bin/netflow2ng --listen 0.0.0.0:2055 --listen-zmq "tcp://*:5556"

suw 26 12:06:26 ntopser systemd[1]: Started netflow2ng - NetFlow v9/IPFIX proxy for ntopng.
suw 26 12:06:26 ntopser netflow2ng[65402]: level=info msg="Using ntopng TLV format for ZMQ"
suw 26 12:06:26 ntopser netflow2ng[65402]: level=info msg="Started ZMQ listener on: tcp://*:5556"
suw 26 12:06:27 ntopser netflow2ng[65402]: level=info msg="Starting collection. It may take several minutes for the first flows to appear"
lines 1-14/14 (END)
```

➤ Étape 5 — Configuration de ntopng en collecteur ZMQ

Le fichier de configuration de ntopng est modifié pour se connecter au flux ZMQ publié par netflow2ng, au lieu d'une écoute NetFlow directe :

/etc/ntopng/ntopng.conf

-i=tcp://127.0.0.1:5556

--http-port=3000

--data-dir=/var/lib/ntopng


```
root@ntopser:~# nano /etc/ntopng/ntopng.conf
```

```
#  
#--ndpi-protocols=/etc/ntopng/custom_protocols.txt  
-i=tcp://127.0.0.1:5556  
--http-port=3000  
--data-dir=/var/lib/ntopng
```

```
sudo systemctl enable ntopng
```

```
sudo systemctl restart ntopng
```

```
sudo journalctl -u ntopng -n 20 --no-pager
```

```
root@ntopser:~# systemctl enable ntopng
```

```
root@ntopser:~# systemctl restart ntopng
```

```
root@ntopser:~# journalctl -u ntopng -n 20 --no-pager
```

Un démarrage réussi se traduit par les lignes « **Started polling on interface 'tcp://127.0.0.1:5556' »** et « **Collecting flows on tcp://127.0.0.1:5556 »** dans les journaux du service.

Génération et export des flux depuis client-test

Sur la VM **client-test**, l'outil open source **softflowd** capture le trafic réseau sortant et l'exporte en **NetFlow v9** vers **netflow2ng**, sans nécessiter de licence ni de matériel réseau dédié :

```
sudo apt update
```

```
root@ntopser:~# apt update
```

```
sudo apt install -y softflowd
```

```
root@clienttest:~# apt install -y softflowd
```

```
sudo softflowd -i ens37 -n 192.168.10.10:2055 -v 9
```

```
root@clienttest:~# softflowd -i ens37 -n 192.168.10.10:2055 -v 9
```

Astuce : le nom de l'interface réseau (ens37 dans cet exemple) doit être adapté selon le nommage réel attribué par VMware Workstation à l'adaptateur connecté à VMnet2, vérifiable avec la commande `ip a`.

Validation de la réception des flux

La réception des paquets **UDP** peut être vérifiée indépendamment de **ntopng**, directement sur **ntop-srv**, avant de vérifier l'affichage final :

```
sudo tcpdump -i ens34 udp port 2055 -n -c 5
```

```
09:35:01.421767 IP 192.168.10.50.38089 > 192.168.10.10.2055: UDP, length 440
```

```
09:35:01.421767 IP 192.168.10.50.38089 > 192.168.10.10.2055: UDP, length 440
```

```
5 packets captured
```

```
5 packets received by filter
```

```
0 packets dropped by kernel
```

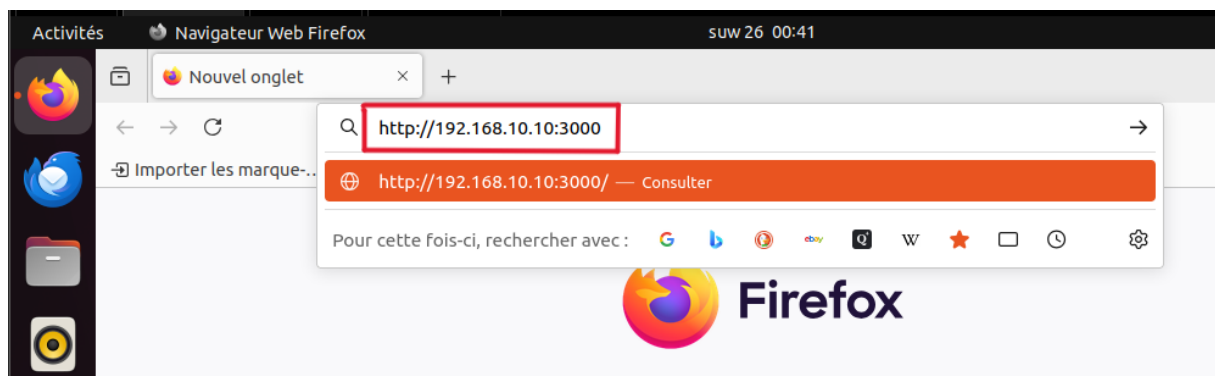
```
root@ntopser:~# tcpdump -i ens34 udp port 2055 -n -c 5
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens34, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:35:01.421767 IP 192.168.10.50.38089 > 192.168.10.10.2055: UDP, length 440
09:35:01.421767 IP 192.168.10.50.38089 > 192.168.10.10.2055: UDP, length 440
09:35:01.421767 IP 192.168.10.50.38089 > 192.168.10.10.2055: UDP, length 440
09:35:01.421767 IP 192.168.10.50.38089 > 192.168.10.10.2055: UDP, length 440
09:35:01.421767 IP 192.168.10.50.38089 > 192.168.10.10.2055: UDP, length 440
5 packets captured
5 packets received by filter
0 packets dropped by kernel
```

Cette capture confirme que **softflowd** exporte effectivement des datagrammes **NetFlow** vers ntop-srv. Un trafic de test généré par **ping** ou **curl** devient alors visible dans l'onglet « **Flows** » de ntopng (<http://192.168.10.10:3000>) en quelques secondes, confirmant le fonctionnement de bout en bout de la chaîne complète.

ping

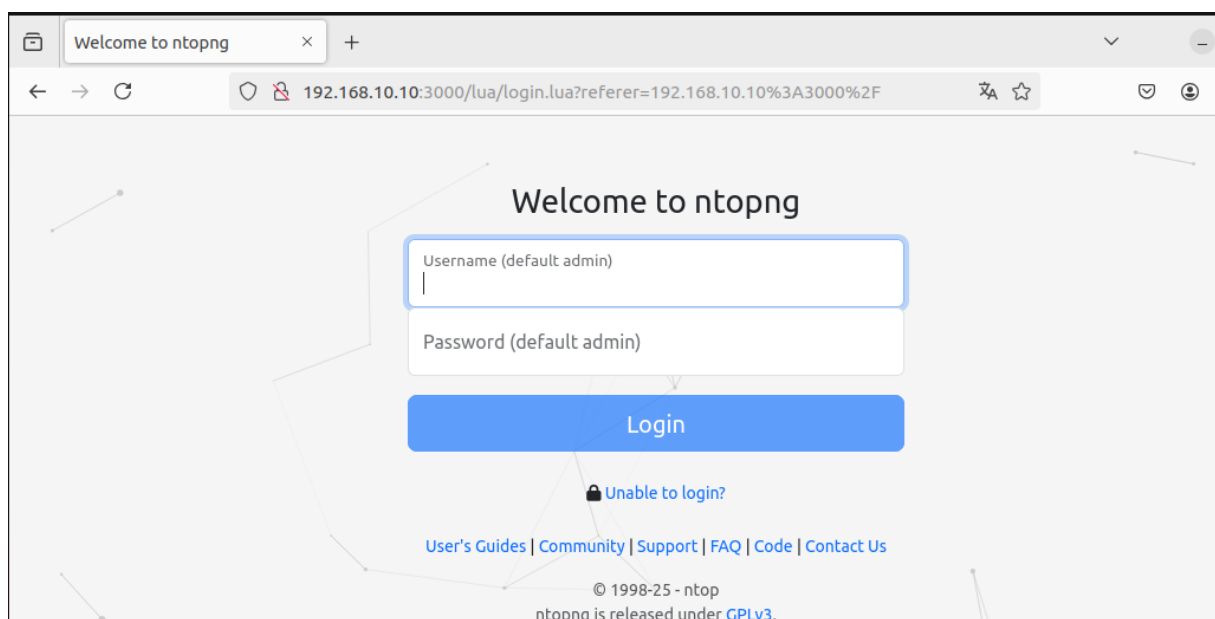
```
root@clienttest:~# ping -c 20 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data.
64 bytes from 192.168.10.10: icmp_seq=1 ttl=64 time=0.885 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=64 time=0.761 ms
64 bytes from 192.168.10.10: icmp_seq=3 ttl=64 time=2.82 ms
64 bytes from 192.168.10.10: icmp_seq=4 ttl=64 time=1.06 ms
64 bytes from 192.168.10.10: icmp_seq=5 ttl=64 time=0.960 ms
64 bytes from 192.168.10.10: icmp_seq=6 ttl=64 time=0.562 ms
64 bytes from 192.168.10.10: icmp_seq=7 ttl=64 time=1.17 ms
64 bytes from 192.168.10.10: icmp_seq=8 ttl=64 time=0.942 ms
64 bytes from 192.168.10.10: icmp_seq=9 ttl=64 time=1.09 ms
64 bytes from 192.168.10.10: icmp_seq=10 ttl=64 time=0.802 ms
64 bytes from 192.168.10.10: icmp_seq=11 ttl=64 time=0.976 ms
64 bytes from 192.168.10.10: icmp_seq=12 ttl=64 time=0.918 ms
64 bytes from 192.168.10.10: icmp_seq=13 ttl=64 time=0.483 ms
64 bytes from 192.168.10.10: icmp_seq=14 ttl=64 time=0.741 ms
64 bytes from 192.168.10.10: icmp_seq=15 ttl=64 time=0.367 ms
64 bytes from 192.168.10.10: icmp_seq=16 ttl=64 time=1.04 ms
64 bytes from 192.168.10.10: icmp_seq=17 ttl=64 time=0.882 ms
64 bytes from 192.168.10.10: icmp_seq=18 ttl=64 time=0.477 ms
64 bytes from 192.168.10.10: icmp_seq=19 ttl=64 time=1.01 ms
64 bytes from 192.168.10.10: icmp_seq=20 ttl=64 time=1.08 ms
--- 192.168.10.10 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19125ms
rtt min/avg/max/mdev = 0.367/0.951/2.822/0.482 ms
root@clienttest:~#
```

L'onglet « Flows » de ntopng (**http://192.168.10.10:3000**)

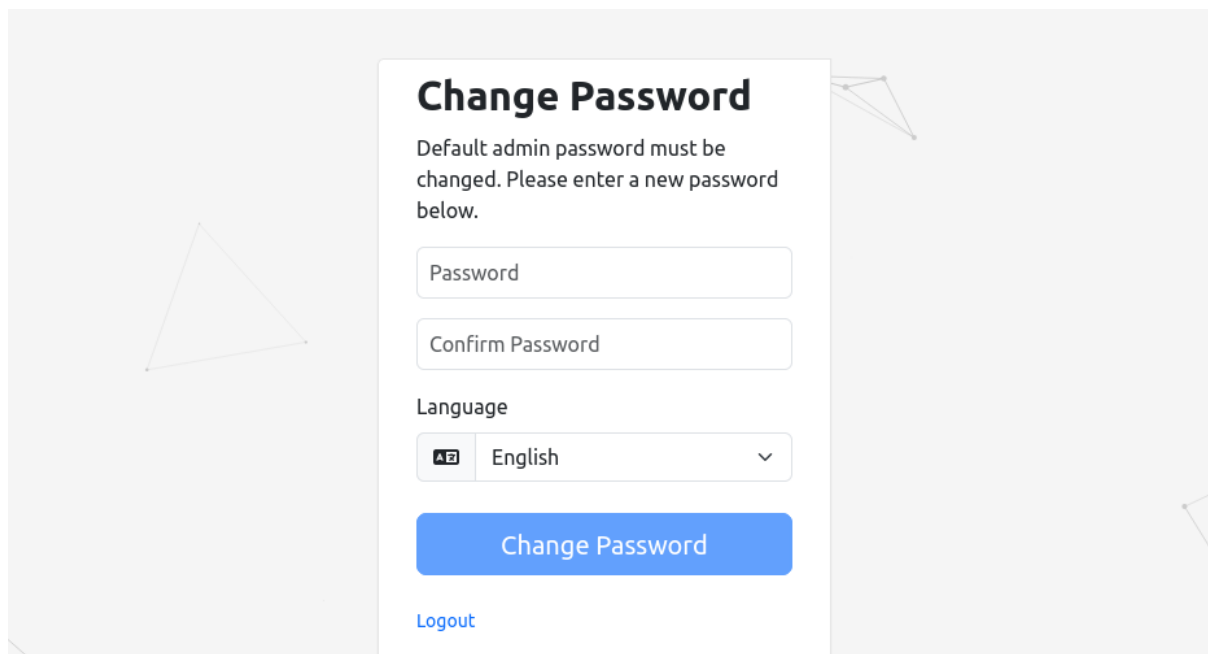


Connecte-toi avec :

- Username : **admin**
- Password : **admin**

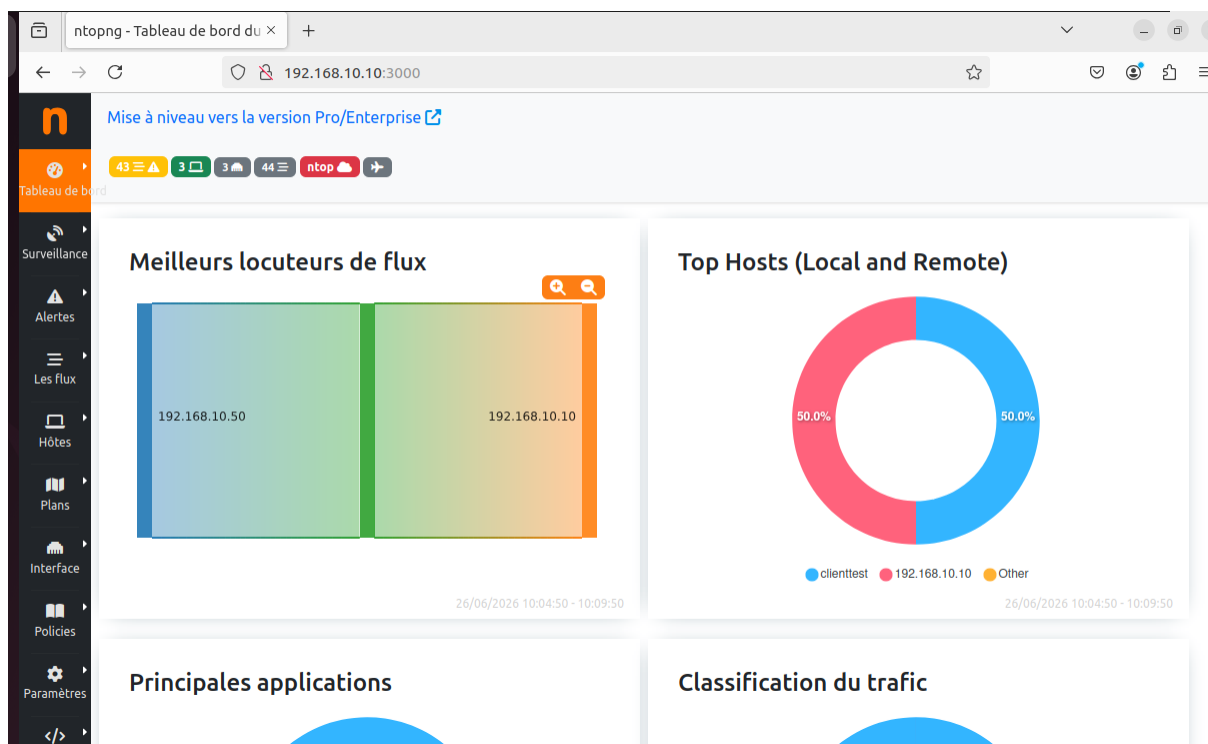


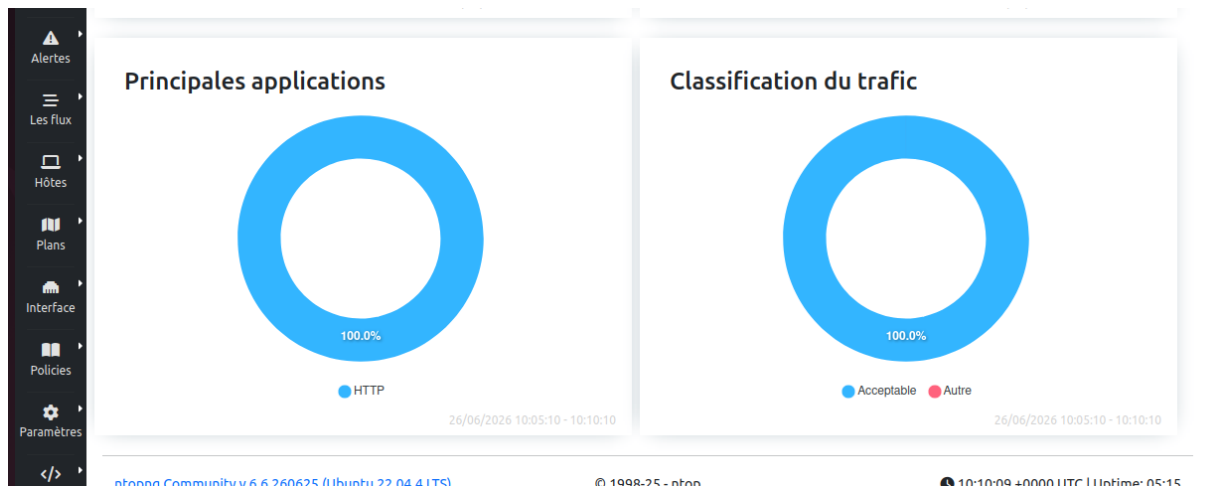
Important : change ce mot de passe par défaut dès la première connexion (ntopng te le proposera probablement automatiquement, sinon va dans **Settings** → **Users** une fois connecté).



The image shows a 'Change Password' form. It has a title 'Change Password' and a message: 'Default admin password must be changed. Please enter a new password below.' There are two input fields: 'Password' and 'Confirm Password'. Below these is a 'Language' section with a dropdown menu currently set to 'English'. At the bottom is a blue button labeled 'Change Password' and a link labeled 'Logout'.

ntopng tourne et tu as accès au **dashboard**





Installation et configuration de la pile ELK

➤ Étape 1 — Installation d'Elasticsearch

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | \
sudo gpg --dearmor -o /usr/share/keyrings/elastic.gpg
echo "deb [signed-by=/usr/share/keyrings/elastic.gpg] \
https://artifacts.elastic.co/packages/8.x/apt stable main" | \
sudo tee /etc/apt/sources.list.d/elastic-8.x.list
```

```
root@elkser:~# wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | \
sudo gpg --dearmor -o /usr/share/keyrings/elastic.gpg
echo "deb [signed-by=/usr/share/keyrings/elastic.gpg] \
https://artifacts.elastic.co/packages/8.x/apt stable main" | \
> tee /etc/apt/sources.list.d/elastic-8.x.list
deb [signed-by=/usr/share/keyrings/elastic.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main
root@elkser:~#
```

```
sudo apt update && sudo apt install -y elasticsearch
```

```
root@elkser:~# apt update && sudo apt install -y elasticsearch
```

```
sudo systemctl enable --now elasticsearch
```

```
root@elkser:~# systemctl enable --now elasticsearch
Created symlink /etc/systemd/system/multi-user.target.wants/elasticsearch.service → /lib/systemd/system/elasticsearch.service.
root@elkser:~#
```

➤ Étape 2 — Installation de Logstash et pipeline NetFlow

```
sudo apt install -y logstash
```

```
root@elkser:~# apt install -y logstash
```

```
sudo /usr/share/logstash/bin/logstash-plugin install logstash-input-netflow
```

```
root@elkser:~# /usr/share/logstash/bin/logstash-plugin install logstash-input-netflow
```

Le pipeline est défini dans **/etc/logstash/conf.d/netflow.conf** :

```
root@elkser:~# nano /etc/logstash/conf.d/netflow.conf
```

```
input {  
  udp {  
    port => 9995  
    codec => netflow { versions => [5, 9, 10] }  
  }  
}  
output {  
  elasticsearch {  
    hosts => ["https://192.168.10.20:9200"]  
    index => "netflow-%{+YYYY.MM.dd}"  
  }  
}
```

```
GNU nano 6.2 /etc/logstash/conf.d/netflow.conf *  
input {  
  udp {  
    port => 9995  
    codec => netflow { versions => [5, 9, 10] }  
  }  
}  
output {  
  elasticsearch {  
    hosts => ["https://192.168.10.20:9200"]  
    index => "netflow-%{+YYYY.MM.dd}"  
  }  
}
```

➤ Étape 3 — Installation de Kibana

```
sudo apt install -y kibana
```

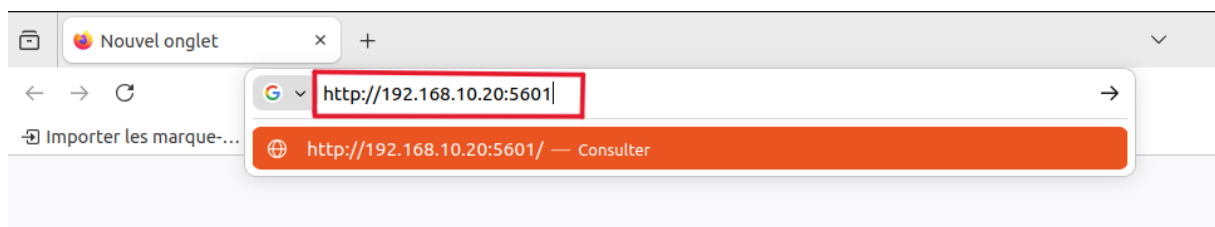
```
root@elkser:~# apt install -y kibana
```

```
sudo systemctl enable --now kibana
```

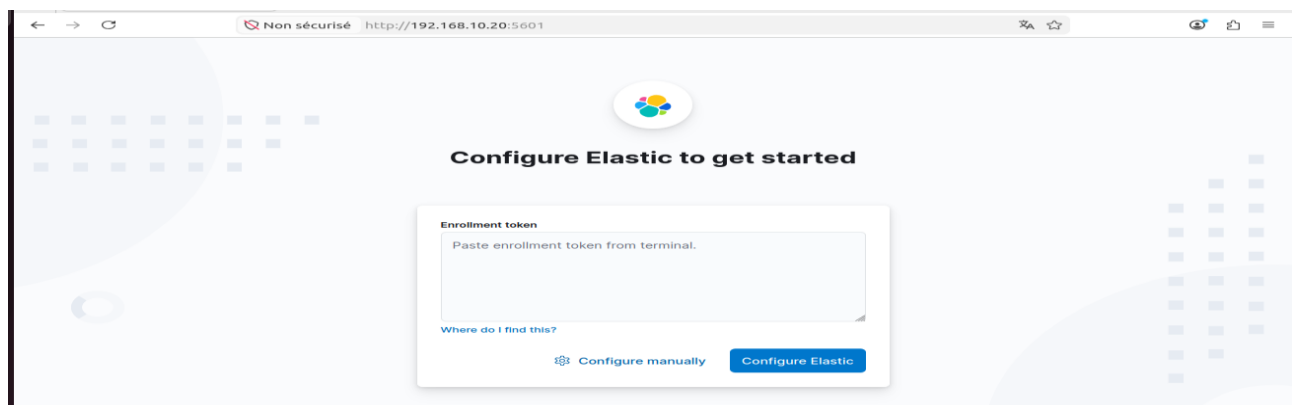
```
root@elkser:~# systemctl enable --now kibana
```

Une fois les premiers documents indexés (**visible via Stack Management > Index Management**), la vue **Discover** permet d'explorer librement les flux, puis de construire des visualisations : volume d'octets par protocole dans le temps, top 10 des adresses IP sources par volume, répartition géographique des connexions entrantes grâce à l'enrichissement **GeoIP** appliqué par **Logstash**.

L'interface Kibana est accessible via <http://192.168.10.20:5601>



Kibana te demande maintenant un "**Enrollment token**" (jeton d' enrôlement), pas juste un code simple. C'est normal pour cette version 8.x : il faut générer ce jeton **côté serveur Elasticsearch**, pas le deviner.



Génère le jeton d' enrôlement sur elk-srv

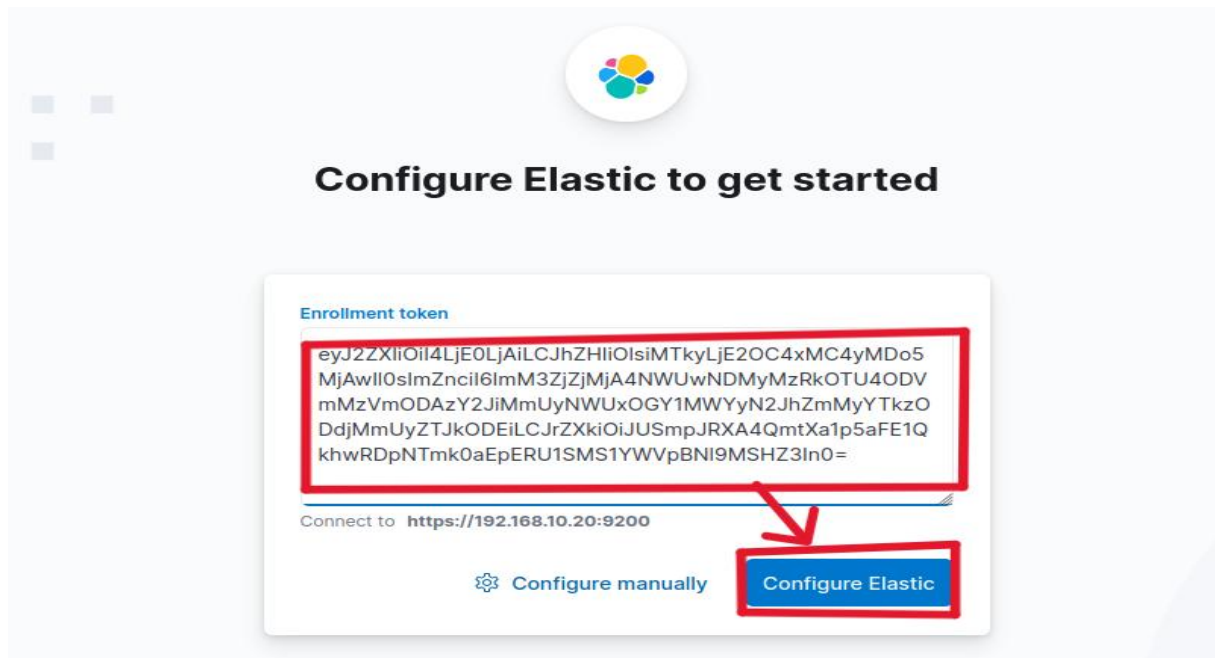
Sur **elk-srv**, lance cette commande (côté Elasticsearch) :

```
root@elkser:~# /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
ey3Z2Xi0i014LjE0LjAiLCJhZHI0LStMTkYlJ3E2OC4xM04yMD05MjAwI0s1mZnci16ImM3ZjZjMjA4NWUwNDMyZmRkOTU4ODVmMzVmODAzY2JlMmUu
yNWUxOGY1MWYyNDJhZmM5YTZkODQdjmUyYjZkODElLCR3Xk0iLjUJUSpmJkXA4QmTxa1p5aFE1QkhWRDpNTmk0aEaPERU1SMS1YWVpBNl9MSHZ3In0=
root@elkser:~#
```

Ça va t'afficher une longue chaîne de caractères (le token). **Copie-la entièrement.**

Colle ce token dans le formulaire Kibana

Retourne sur ton navigateur, dans le champ "**Enrollment token**", colle exactement ce que la commande a affiché, puis valide.



Petite subtilité : Kibana est en attente de configuration interactive

Il n'a pas encore généré de nouveau code (**code=XXXXXX**), il est en train de "Valider la connexion Elasticsearch" (**Validating Elasticsearch connection configuration...**). Attends quelques secondes/minute, puis regarde la suite des logs :

```
root@elkser:~# journalctl -u kibana -n 10 --no-pager
```

Tu devrais voir apparaître une nouvelle ligne avec un lien et un code, cette fois avec l'IP réseau correcte, du type :

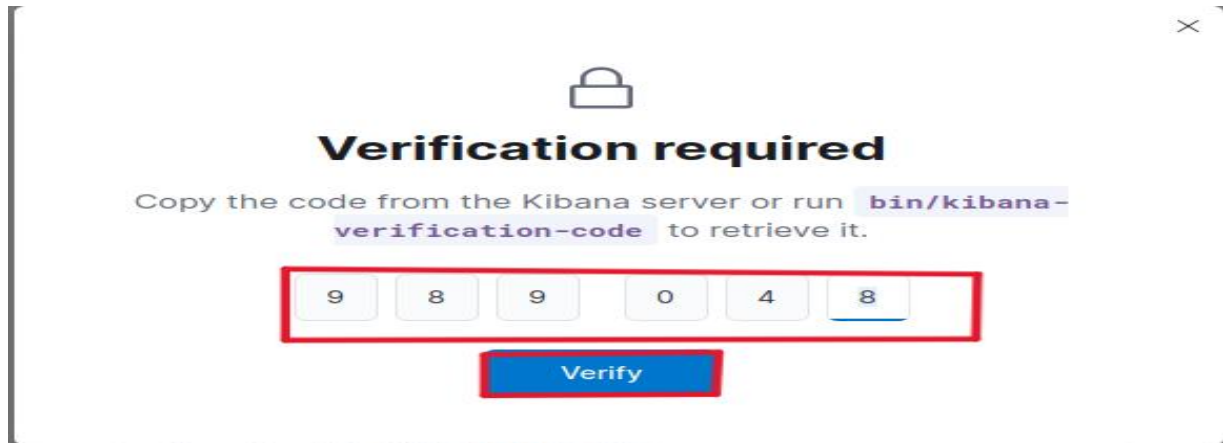
Go to <http://192.168.10.20:5601/?code=989048> to get started.

Pendant ce temps, vérifie qu'Elasticsearch tourne bien (Kibana en dépend)

```
root@elkser:~# systemctl status elasticsearch
```


Une fois le nouveau code affiché, va sur ton navigateur

Utilise le code affiché dans les logs pour terminer la configuration initiale (association à ton cluster Elasticsearch).



Il reste juste le mot de passe **elastic** (tu n'as toujours pas réinitialisé le bon, ou tu as retapé l'ancien par erreur). Réglons ça maintenant que le disque n'est plus un obstacle.

Réinitialise le mot de passe elastic (le cluster est sain maintenant, ça va marcher)

```
root@elkser:~# /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic
This tool will reset the password of the [elastic] user to an autogenerated value.
The password will be printed in the console.
Please confirm that you would like to continue [y/N]y

Password for the [elastic] user successfully reset.
New value: qXVf9uChIQjIi06baEtB
```

Copie précisément le nouveau mot de passe affiché.

Vérifie la santé du cluster avec ce nouveau mot de passe

Tape le nouveau mot de passe quand demandé. Tu devrais voir "**status**": "**green**"

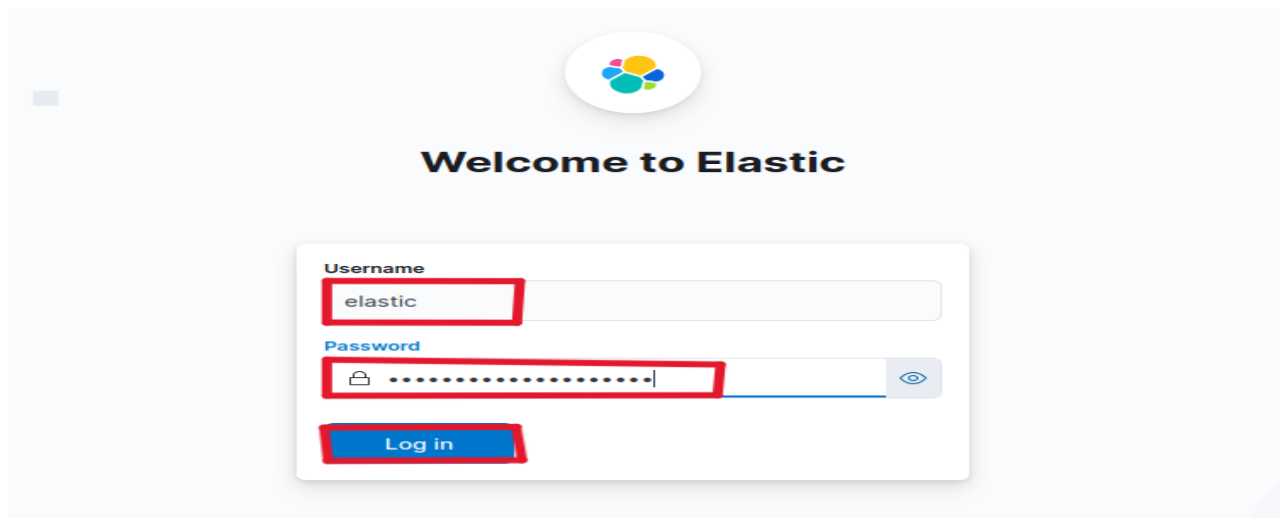
```
root@elkser:~# curl -k -u elastic https://localhost:9200/_cluster/health?pretty
Enter host password for user 'elastic':
{
  "cluster_name" : "elasticsearch",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 1,
  "number_of_data_nodes" : 1,
  "active_primary_shards" : 39,
  "active_shards" : 39,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 0,
  "unassigned_primary_shards" : 0,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 100.0
}
```

Dernière étape : vérifie Kibana

Retourne sur ton navigateur :

Connecte-toi

- **Username** : **elastic**
- **Password** : celui que tu as récupéré avec **elasticsearch-reset-password** (celui qui a fonctionné pour le curl juste avant)



Dernière étape pour boucler la chaîne complète

Il reste à connecter le flux NetFlow vers Logstash aussi (pas seulement ntopng), pour que Kibana affiche des données :

1. Démarre Logstash (si pas déjà fait)

```
sudo systemctl enable --now kibana
```

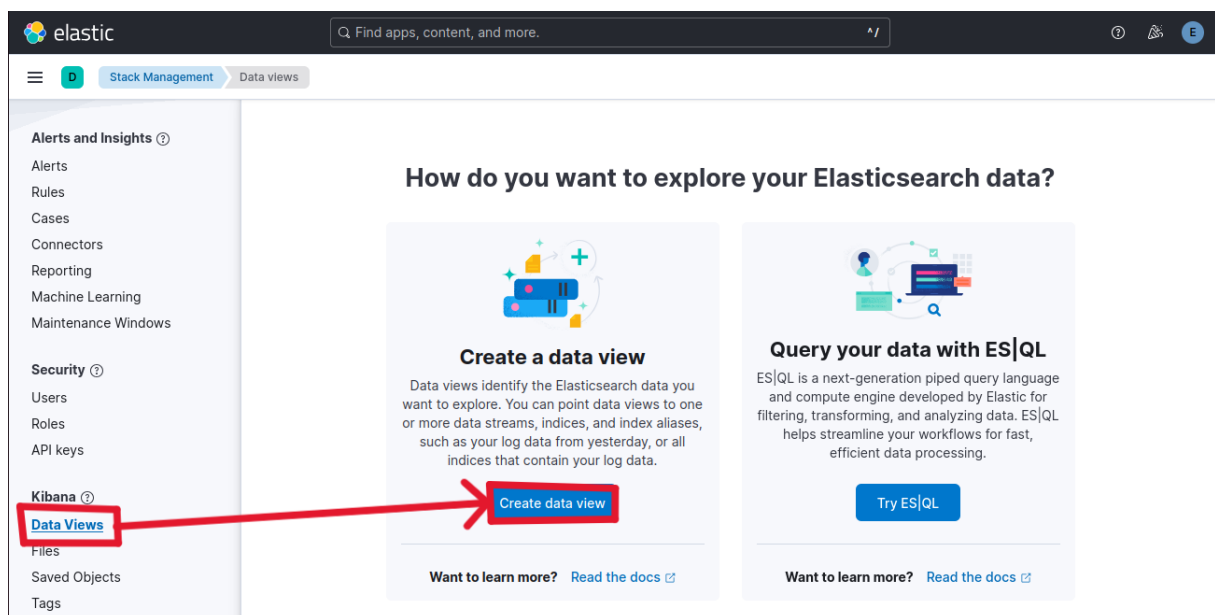
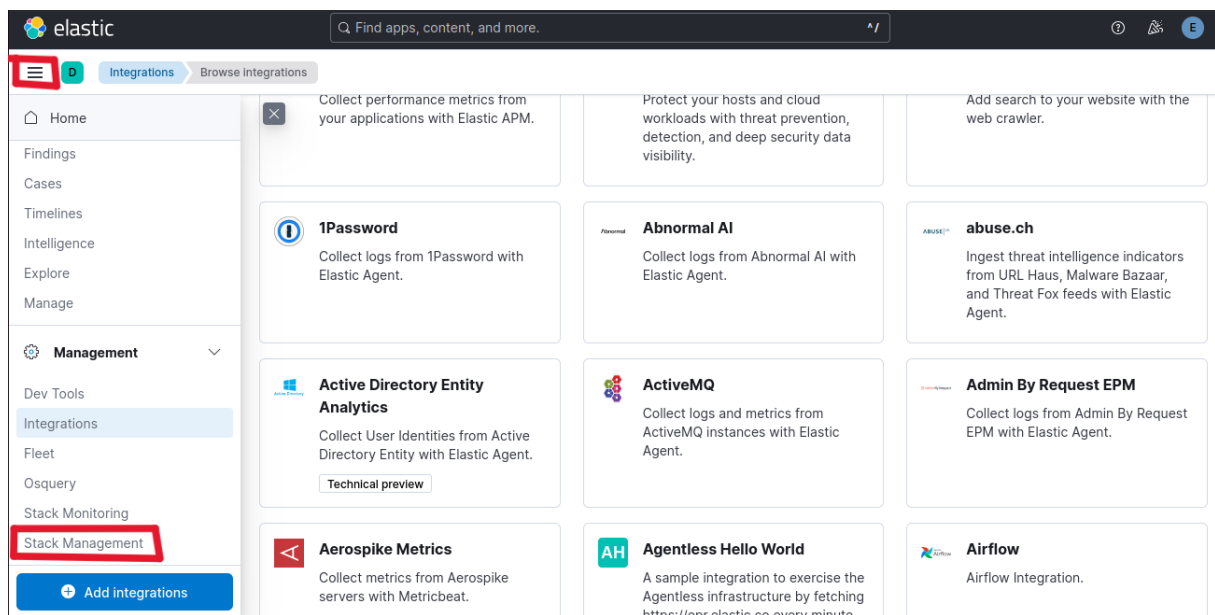
```
root@elkser:~# systemctl enable --now kibana
```

```
sudo systemctl status logstash
```

```
root@elkser:~# systemctl status logstash
```

2. Crée un Data View dans Kibana

Dans Kibana : menu ☰ → **Stack Management** → **Data Views** → **Create data view** → motif **netflow-***



Menu ☰ → Stack Management → Data Views → Create data view

- Name : **netflow-***
- Index pattern : **netflow-***
- Timestamp field : sélectionne le champ approprié (probablement **@timestamp**)

Create data view

Name

Index pattern

Timestamp field

Select a timestamp field for use with the global time filter.
[Show advanced settings](#)

× Close [Save data view to Kibana](#)

✓ Your index pattern matches 1 source.

All sources Matching sources

netflow-2026.06.29 Index

Rows per page: 10

netflow-*

Delete Edit

Index pattern: netflow-* Time field: @timestamp ★ Default

Fields (37) Scripted fields (0) Field filters (0) Relationships (0)

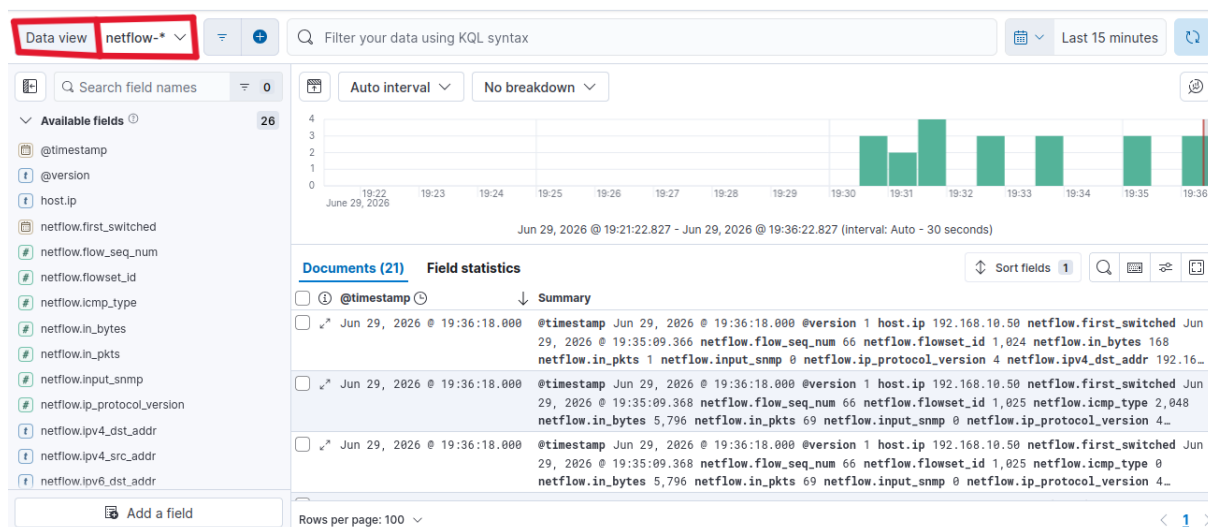
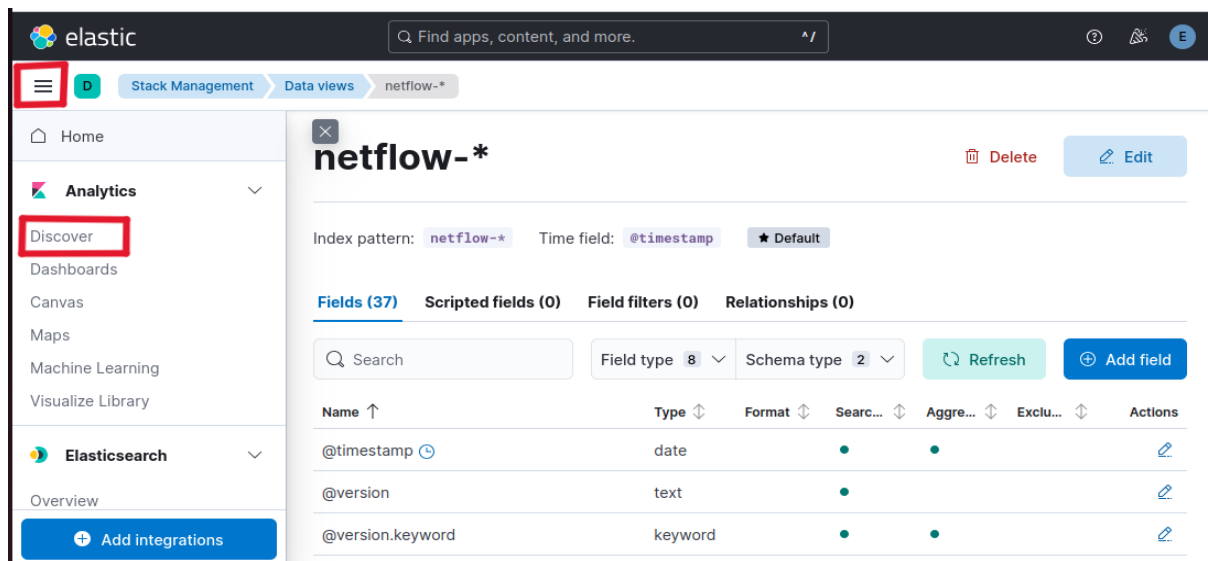
Search

Field type 8 Schema type 2 Refresh Add field

Name ↑	Type ↑	Format ↑	Searchable ↑	Aggregata... ↑	Excluded ↑	Actions
@timestamp	date		•	•		Edit
@version	text		•			Edit
@version.keyword	keyword		•	•		Edit
_id	_id		•			Edit
_ignored	_ignored		•	•		Edit
_index	_index		•	•		Edit

3. Génère du trafic et vérifie dans Discover

Menu ☰ → **Discover** → sélectionne ta data view **netflow-***



Export via GNS3 (routeur et switch réels)

En complément de la chaîne softflowd/netflow2ng, le laboratoire prévoit l'intégration d'un véritable routeur Cisco IOS et d'un switch **Open vSwitch** virtualisés via **GNS3**, reliés aux VM VMware Workstation via le réseau **VMnet Host-only (VMnet2)**, afin de représenter un déploiement réseau plus proche d'un environnement de production.

➤ Étape 1 — Interconnexion GNS3 / VMware Workstation

- Créer un réseau Host-only dédié dans VMware (**Edit > Virtual Network Editor > Add Network**), par exemple VMnet2, avec le service DHCP désactivé et la plage 192.168.10.0/24.
- Ajouter un second adaptateur réseau sur chaque VM (paramètres VM > Network Adapter > Custom), pointant vers ce même VMnet2.
- Dans GNS3, ajouter un nœud Cloud, l'éditer et sélectionner VMnet2 dans la liste des interfaces réseau disponibles sur la machine hôte, puis y relier le routeur et le switch.

➤ Étape 2 — Configuration du routeur Cisco (NetFlow v9)

```
Router(config)# interface FastEthernet0/0
Router(config-if)# ip address 192.168.10.1 255.255.255.0
Router(config-if)# no shutdown
Router(config)# ip flow-export destination 192.168.10.10 2055
Router(config)# ip flow-export version 9
Router(config)# interface FastEthernet0/0
Router(config-if)# ip flow ingress
Router(config-if)# ip flow egress
```

```
R1#conf t
Enter configuration commands, one per line. End with CNTL/Z.
R1(config)#interface FastEthernet0/1
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#no sh
R1(config-if)#no shutdown
R1(config-if)#
*Mar 1 00:02:58.359: %LINK-3-UPDOWN: Interface FastEthernet0/1, changed state to up
*Mar 1 00:02:59.359: %LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to up
R1(config-if)#exit
R1(config)#ip flow-export destination 192.168.10.10 2055
R1(config)#ip flow-export version 9
R1(config)#interface FastEthernet0/2
^
% Invalid input detected at '^' marker.

R1(config)#interface FastEthernet0/0
R1(config-if)#ip flow ingress
R1(config-if)#ip flow egress
R1(config-if)#
```

➤ Étape 3 — Configuration du switch (sFlow)

```
ovs-vsctl -- --id=@sflow create sflow agent=eth0 \
target="\192.168.10.10:6343\" sampling=64 polling=10 \
-- set bridge br0 sflow=@sflow
```

```
root@elkser:~# ovs-vsctl -- --id=@sflow create sflow agent=eth0 \
target="\192.168.10.10:6343\" sampling=64 polling=10 \
-- set bridge br0 sflow=@sflow
```

Avertissement : le taux d'échantillonnage sFlow (paramètre sampling) doit être choisi avec soin : une valeur trop élevée peut masquer des flux de faible volume significatifs pour la détection de sécurité. Une valeur de 1/64 à 1/512 est un point de départ raisonnable.

Les flux exportés par le routeur et le switch GNS3 sont reçus par la même instance **netflow2ng** que celle utilisée pour softflowd, ce qui permet de mutualiser un point de collecte unique pour l'ensemble des sources de flux du laboratoire, qu'elles soient logicielles ou matérielles.

VI. Résultats et tests

Afin de valider le bon fonctionnement de la chaîne complète, plusieurs scénarios de test ont été exécutés depuis la VM client-test, en utilisant des outils standards : **iperf3** pour la bande passante, **curl** et **wget** pour le trafic HTTP/HTTPS, et **nmap** pour simuler un balayage de ports.

Scénario	Outil / méthode	Résultat observé
Trafic volumétrique	iperf3 en mode TCP, 5 minutes	Flux visibles en moins de 60 s dans ntopng ; agrégation correcte dans Kibana
Identification applicative	Navigation web, DNS, SSH	nDPI identifie correctement HTTP, HTTPS (SNI), DNS et SSH
Balayage de ports	nmap -sS sur 192.168.10.0/24	Pic de flux courts détecté ; alerte ntopng « scan détecté »
Comparaison NetFlow/sFlow	Même trafic observé via les deux exports	Comptage NetFlow exact ; comptage sFlow proche avec variance liée à l'échantillonnage
Panne simulée ELK	Arrêt du service Logstash pendant 10 min	ntopng continue de fonctionner normalement ; rattrapage à la reprise

Tableau 6 — Synthèse des scénarios de test et résultats observés

Indicateurs de performance observés

Sur l'environnement de laboratoire, après une période d'observation continue de 48 heures, les indicateurs suivants ont été relevés :

- **Latence d'apparition d'un flux dans ntopng** : de l'ordre de quelques secondes après son export.
- **Latence d'indexation dans Elasticsearch** : également de quelques secondes, débit d'ingestion largement suffisant pour le volume testé.
- **Charge CPU de softflowd / ntopng** : restée inférieure à 5 % sur un cœur en régime nominal.
- **Volume de stockage** : environ 1,2 Go d'index Elasticsearch générés pour 48 heures de trafic de test modéré.

Astuce : le test de panne de la chaîne ELK confirme la résilience de l'architecture : la duplication des flux à la source (cf. § Architecture) garantit qu'une panne d'un composant n'affecte pas la disponibilité de l'autre chaîne.

VII. Sécurité et détection d'anomalies

Au-delà de la supervision de performance, l'analyse de flux constitue un pilier reconnu de la **détection d'incidents de sécurité** réseau. De nombreux comportements malveillants se traduisent par des signatures observables dans les métadonnées de flux, sans qu'il soit nécessaire d'inspecter le contenu des paquets.

Comportement	Signature dans les flux
Balayage de ports	Nombreux flux très courts vers de nombreux ports distincts d'une même IP
Exfiltration de données	Volume sortant anormalement élevé vers une destination externe rarement contactée
Beaconing C2	Connexions de faible volume, très régulières dans le temps, vers une même destination
Attaque DDoS	Pic soudain du nombre de flux et du volume entrant depuis de nombreuses sources distinctes

Tableau 3 — Typologie des comportements malveillants détectables par analyse de flux

Test réalisé en laboratoire

Le scénario testé utilise **nmap** pour simuler un balayage de ports vers une machine du laboratoire. Chaque tentative de connexion vers un port différent génère un nouveau flux distinct, produisant un pic caractéristique du nombre de flux par seconde.

- **ntopng** détecte ce pic via son moteur de scoring comportemental et déclenche une alerte « possible port scan ».
- **Kibana** permet de retrouver et documenter le même événement a posteriori, via une visualisation « Top N » sur l'adresse IP source.

Avertissement : avec sFlow, un balayage de ports utilisant un faible nombre de paquets peut, selon le taux d'échantillonnage, rester partiellement invisible. NetFlow/IPFIX (exhaustif) reste préférable à sFlow pour les segments réseau à enjeu de sécurité élevé.

Apports et limites observés

Apports	Limites
Visibilité immédiate et granulaire du trafic	Complexité de déploiement de la chaîne ELK
Détection comportementale sans signature préalable	Imprécision statistique inhérente à sFlow
Résilience : panne ELK sans impact sur ntopng	Volumétrie de stockage à anticiper (ILM)
Coût maîtrisé (100% open source)	Dépendance à la synchronisation NTP des équipements

Tableau 4 — Synthèse des apports et limites de la solution

Comparaison avec les solutions commerciales

Afin de replacer cette solution dans son contexte concurrentiel, il est utile de la comparer à des outils commerciaux fréquemment rencontrés en entreprise, tels que **SolarWinds NetFlow Traffic Analyzer** ou **PRTG Network Monitor**.

Critère	ELK + NtopNG	Solutions commerciales
Coût de licence	Aucun (open source)	Élevé, par capteur
Prise en main	Modérée	Élevée (interface guidée)
Personnalisation	Très élevée	Modérée
Scalabilité	Très élevée	Bonne, coût croissant

Tableau 5 — Comparaison qualitative avec des solutions commerciales de référence

Le point de bascule économique en faveur de la solution open source se situe généralement à mesure que le nombre de capteurs ou le volume de flux supervisés augmente : les solutions commerciales facturant par capteur ou par flux par seconde, leur coût croît de façon quasi linéaire avec la taille du réseau, alors que le coût marginal d'extension d'une architecture **ELK/NtopNG** reste principalement lié à l'infrastructure serveur.

Conformité et aspects réglementaires

La collecte de métadonnées de flux réseau, bien que moins intrusive que la capture de paquets complète, reste soumise à des obligations réglementaires, notamment au regard de la protection des données personnelles (une adresse IP pouvant être considérée comme une donnée à caractère personnel au sens du **RGPD**). Plusieurs précautions doivent être intégrées dès la conception :

- Informer les utilisateurs de l'existence d'un dispositif de supervision réseau, conformément aux obligations de transparence applicables en entreprise.
- Limiter la durée de conservation des données de flux à ce qui est strictement nécessaire à la finalité poursuivie, et la documenter dans une politique de conservation.
- Restreindre l'accès aux tableaux de bord et aux données brutes aux seules personnes habilitées, via une gestion fine des rôles dans Kibana et ntopng.

VIII. Dimensionnement et bonnes pratiques

Le dimensionnement d'une chaîne de supervision de flux dépend directement du débit réseau à superviser et donc du nombre de **flux par seconde (flows per second, fps)** générés. Le tableau suivant donne des ordres de grandeur indicatifs, à ajuster selon le contexte réel par des tests de charge.

Profil réseau	Débit agrégé	Flux/s estimés	vCPU	RAM
Petite entreprise	< 100 Mbit/s	< 500 fps	2-4	4-8 Go
Moyenne entreprise	100 Mbit/s - 1 Gbit/s	500 - 5 000 fps	4-8	16-32 Go
Datacenter / opérateur	> 10 Gbit/s	> 20 000 fps	16+ (cluster)	64+ Go (cluster)

Tableau 7 — Ordres de grandeur indicatifs de dimensionnement

Pour le laboratoire de démonstration objet de ce rapport, un profil « petite entreprise » a été retenu, suffisant pour valider la chaîne complète sans nécessiter d'infrastructure matérielle lourde.

Bonnes pratiques de déploiement

Stratégie de duplication des flux

Un point clé de l'architecture est la manière dont un même flux peut alimenter à la fois ntopng et la chaîne ELK, sans dépendance entre les deux. Trois approches sont possibles :

- **Double export depuis l'équipement** : la plupart des routeurs/switches permettent de configurer plusieurs destinations d'export simultanément. C'est l'approche la plus robuste, retenue par défaut dans ce projet.
- **Duplication réseau (port mirroring UDP)** : un répartiteur de paquets recopie chaque datagramme UDP reçu vers plusieurs destinataires.
- **Chaînage applicatif** : nProbe peut être configuré en mode passthrough pour relayer le flux reçu vers un second collecteur.

Synchronisation temporelle

Une désynchronisation **NTP** entre les équipements exportateurs peut fausser l'ordonnancement temporel des flux dans les tableaux de bord, un point souvent sous-estimé lors des déploiements. Il est recommandé de configurer un serveur NTP de référence commun à l'ensemble des équipements et des VM du laboratoire.

Sauvegarde et continuité d'activité

Il est recommandé de documenter et tester un plan de sauvegarde des configurations (pipelines Logstash, tableaux de bord Kibana, scripts ntopng) afin de sécuriser la continuité d'activité de la supervision elle-même, qui constitue un outil critique en cas d'incident de sécurité.

Avertissement : la supervision réseau ne doit jamais devenir elle-même un point de défaillance unique : la documentation des procédures de restauration doit être testée régulièrement, au même titre que celle des autres services critiques de l'infrastructure.

Conclusion

Ce rapport a présenté la mise en place d'une solution de **supervision réseau** complète, combinant l'analyse de flux **NetFlow** et **sFlow** avec deux chaînes de traitement complémentaires : **NtopNG** (associé à sa sonde nProbe) et la suite **ELK**. Cette solution constitue une réponse concrète aux enjeux actuels de visibilité et de sécurisation du trafic réseau au sein des organisations modernes.

L'ensemble des étapes a été réalisé avec succès : compréhension théorique des protocoles, conception de l'architecture, déploiement du laboratoire à trois machines virtuelles, intégration d'un routeur et d'un switch réels via **GNS3**, puis validation par des tests pratiques incluant la détection d'un balayage de ports. La solution est désormais pleinement opérationnelle, avec une visibilité accessible à la fois via **ntopng** (port 3000) et **Kibana** (port 5601).

Sur le plan de la sécurité, cette solution apporte une valeur ajoutée considérable : détection en temps réel de comportements suspects, corrélation des événements, et génération d'alertes automatiques. Sur le plan des performances, l'optimisation des paramètres mémoire, le dimensionnement à trois VM adapté à une contrainte de 16 Go de RAM, et la duplication indépendante des deux chaînes ont permis de garantir un fonctionnement stable et résilient.

En perspective, il serait intéressant d'intégrer cette solution à un **SIEM/SOC** pour la détection avancée, d'automatiser la réponse aux incidents via des outils SOAR, d'exploiter le machine learning pour la détection d'anomalies, et de migrer progressivement vers **IPFIX**, standard ouvert plus pérenne que NetFlow propriétaire.

Recommandations pour un déploiement en production

Sur la base des enseignements du laboratoire, les recommandations suivantes peuvent être formulées pour un passage à l'échelle en environnement professionnel réel :

- **Audit préalable du parc réseau** : identifier les protocoles d'export disponibles (NetFlow v5/v9, IPFIX, sFlow) et planifier les mises à jour de firmware nécessaires.
- **Test de charge représentatif** : dimensionner la chaîne de collecte sur la base d'un débit réel plutôt que sur des estimations théoriques.
- **Politique de cycle de vie des données** : mettre en place dès le départ une politique **ILM** (Index Lifecycle Management) pour éviter toute saturation de stockage à moyen terme.
- **Haute disponibilité** : prévoir une architecture Elasticsearch en cluster (a minima 3 nœuds) pour les déploiements de taille moyenne à grande.
- **Formation des équipes** : former les équipes d'exploitation à l'utilisation conjointe de ntopng (diagnostic immédiat) et de Kibana (analyse approfondie).

Mot de fin

La supervision réseau par analyse de flux ne doit pas être perçue comme un choix exclusif entre outils, mais comme un ensemble de briques complémentaires à assembler selon les besoins réels de l'organisation. Ce travail démontre qu'il est possible, avec des outils **open source** matures et bien documentés, de construire une chaîne de supervision réseau complète, performante et évolutive, sans dépendre de licences commerciales coûteuses.

Annexe — Glossaire et références

Glossaire des termes techniques

Terme	Définition
Flux (flow)	Ensemble de paquets partageant une combinaison commune d'adresses IP, de ports et de protocole, traité comme une unité statistique unique.
IPFIX	IP Flow Information Export — standard IETF d'export de flux dérivé de NetFlow v9 (RFC 7011).
nDPI	Bibliothèque d'inspection profonde de paquets développée par ntop, utilisée pour l'identification des applications.
softflowd	Démon open source capturant le trafic d'une interface et l'exportant en NetFlow, utilisé en laboratoire sur la VM client-test.
netflow2ng	Collecteur NetFlow/IPFIX open source qui reçoit les flux UDP et les republie vers ntopng via un bus ZMQ, alternative gratuite à nProbe.
ZMQ (ZeroMQ)	Bibliothèque de messagerie asynchrone utilisée pour la communication entre netflow2ng et ntopng.
ILM	Index Lifecycle Management — gestion automatisée du cycle de vie des index Elasticsearch.
Top talkers	Hôtes ou applications générant les volumes de trafic les plus importants sur une période donnée.
SOC / SIEM	Security Operations Center / Security Information and Event Management — centre et plateforme de supervision de la sécurité.

Tableau récapitulatif des ports utilisés

Port	Protocole	Usage
2055	UDP	Réception NetFlow v5/v9/IPFIX (netflow2ng)
6343	UDP	Réception sFlow
5556	TCP	Bus ZMQ entre netflow2ng et ntopng
3000	TCP	Interface web ntopng
9200	TCP	API REST Elasticsearch
5601	TCP	Interface web Kibana

Références

- Documentation officielle ntop / ntopng — <https://www.ntop.org>
- Documentation officielle Elastic (Elasticsearch, Logstash, Kibana) — <https://www.elastic.co/guide>
- IETF RFC 7011 — Specification of the IP Flow Information Export (IPFIX) Protocol
- sFlow.org — spécifications du protocole sFlow
- Cisco Systems — documentation NetFlow v5/v9